

ĐẠI HỌC QUỐC GIA TP.HCM
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ THÔNG TIN



LỚP: CS112.Q11.KHTN

MÔN HỌC: PHÂN TÍCH VÀ THIẾT KẾ THUẬT TOÁN

BTVN nhóm 14

Nhóm 4:

Lê Văn Thức

Bảo Quý Định Tân

Giảng viên:

Nguyễn Thanh Sơn

1 Lời giải bài A: LongestPath

1.1 Phân tích phương pháp thiết kế thuật toán

Bài toán tìm đường đi đơn dài nhất (Longest Path Problem) trên đồ thị tổng quát thuộc lớp bài toán **NP-Hard**. Không tồn tại thuật toán đa thức nào để giải chính xác (như Quy hoạch động hay Duyệt đồ thị thông thường) cho mọi trường hợp.

Do đó, giải pháp được đề xuất là một thuật toán lai ghép (Hybrid Algorithm), kết hợp các kỹ thuật thiết kế thuật toán sau:

1. **Backtracking (Quay lui):** Đây là khung sườn chính của thuật toán. Ta sử dụng DFS để duyệt sâu vào các nhánh của đồ thị nhằm xây dựng đường đi. Khi đi vào ngõ cụt, thuật toán quay lui để thử các hướng khác.
2. **Branch and Bound (Nhánh cận Cắt tỉa):** Để tránh việc duyệt vào các nhánh vô nghiệm (không thể về đích), ta sử dụng kỹ thuật cắt tỉa (Pruning) thông qua hàm lookahead.
 - *Cận (Bound):* Trước khi thăm đỉnh v , ta kiểm tra xem từ v có tồn tại đường đi nào về đích N hay không (sử dụng BFS cục bộ hoặc khoảng cách đã tính trước).
 - *Cắt tỉa:* Nếu không tồn tại đường về đích, ta cắt bỏ hoàn toàn nhánh đó, giúp giảm đáng kể không gian tìm kiếm.
3. **Greedy Heuristic (Tham lam Heuristic):** Thay vì chọn đỉnh kề ngẫu nhiên, ta áp dụng tư tưởng tham lam dựa trên Heuristic để chọn nước đi "tốt nhất" tại mỗi bước:
 - *Warnsdorff's Rule:* Ưu tiên đỉnh có bậc khả dụng thấp (ít lối ra) để tránh việc các đỉnh này bị cô lập sau này.
 - *Distance Heuristic:* Ưu tiên đỉnh ở xa đích để dành không gian gần đích cho giai đoạn cuối của đường đi.
4. **Local Search (Tìm kiếm cục bộ):** Đây là kỹ thuật quan trọng để tối ưu hóa kết quả. Sau khi tìm được một đường đi thô, ta áp dụng kỹ thuật *Iterative Improvement* (Cải tiến lặp) bằng cách chèn thêm các đỉnh chưa thăm vào giữa đường đi hiện tại (Triangle Insertion và Box Insertion). Phương pháp này giúp thuật toán thoát khỏi tối ưu cục bộ của DFS.
5. **Randomized Algorithm (Thuật toán ngẫu nhiên):** Để bao phủ không gian tìm kiếm trong giới hạn thời gian ngắn (5 giây), ta sử dụng chiến lược *Restart* (Khởi động lại) liên tục với các tham số trọng số ngẫu nhiên (Randomized Weights). Điều này giúp thuật toán không bị kẹt vào một lối mòn cố định.

1.2 Chi tiết kỹ thuật và Cấu trúc dữ liệu

1.2.1 Hàm mục tiêu Heuristic (Scoring Function)

Tại mỗi bước duyệt, các đỉnh ứng viên v được gán trọng số theo công thức:

$$Score(v) = (Deg(v) \times w_{deg}) - (Dist(v) \times w_{dist}) + Noise$$

Trong đó:

- $Deg(v)$: Số lượng đỉnh kề chưa thăm của v .
- $Dist(v)$: Khoảng cách ngắn nhất từ v đến đích (tính bằng BFS ngược).
- w_{deg}, w_{dist} : Các hệ số trọng số thay đổi động theo từng lần chạy lại.
- $Noise$: Một giá trị ngẫu nhiên nhỏ để phá vỡ sự cân bằng khi các đỉnh có điểm số bằng nhau.

Đỉnh có $Score$ càng nhỏ sẽ được ưu tiên duyệt trước.

1.2.2 Tối ưu hóa Hậu xử lý (Post-processing Optimization)

Ta sử dụng Ma trận kề dạng **Bitmask** để thực hiện các thao tác kiểm tra kề nhau trong $O(1)$. Quy trình tối ưu gồm 2 pha:

- **Pha 1 (Triangle Insertion)**: Với cạnh (u, v) trên đường đi, tìm đỉnh k chưa thăm sao cho (u, k) và (k, v) là cạnh. Chèn k vào giữa: $u \rightarrow k \rightarrow v$.
- **Pha 2 (Box Insertion)**: Tìm cặp đỉnh (a, b) chưa thăm để chèn vào giữa (u, v) tạo thành $u \rightarrow a \rightarrow b \rightarrow v$.

1.3 Đánh giá độ phức tạp

- **Độ phức tạp thời gian**: Thuật toán chạy theo cơ chế *Time-bound*. Gọi T_{limit} là thời gian giới hạn. Độ phức tạp thực tế là $O(T_{limit} \times speed)$, trong đó tốc độ xử lý phụ thuộc vào việc cắt tỉa nhánh. Với $N = 50$, thuật toán có thể thực hiện hàng chục nghìn lần thử lại.
- **Độ phức tạp không gian**:
 - Ma trận kề Bitmask: $O(N^2 / \text{word_size}) \approx O(1)$ với $N = 50$.
 - Danh sách kề: $O(N + M)$.
 - Đệ quy stack: $O(N)$.
 - Tổng cộng: $O(N + M)$.

1.4 Hiện thực (Code Python)

```

1 import sys
2 import time
3 import random
4 from collections import deque
5
6 # Tang giới hạn đệ quy
7 sys.setrecursionlimit(10000)
8
9 def solve():
10     # --- DOC INPUT ---
11     input_data = sys.stdin.read().split()
12     if not input_data: return
13     iterator = iter(input_data)
14
15     try:

```

```

16         N = int(next(iterator))
17         M = int(next(iterator))
18     except StopIteration: return
19
20     adj = [[] for _ in range(N)]
21     adj_mask = [0] * N
22
23     for _ in range(M):
24         u = int(next(iterator)) - 1
25         v = int(next(iterator)) - 1
26         adj[u].append(v)
27         adj[v].append(u)
28         adj_mask[u] |= (1 << v)
29         adj_mask[v] |= (1 << u)
30
31     target = N - 1
32     start_node = 0
33
34     # --- PRE-CALC DISTANCE (BFS) ---
35     dist_from_target = [-1] * N
36     dist_from_target[target] = 0
37     q = deque([target])
38     while q:
39         u = q.popleft()
40         for v in adj[u]:
41             if dist_from_target[v] == -1:
42                 dist_from_target[v] = dist_from_target[u] + 1
43                 q.append(v)
44
45     if dist_from_target[start_node] == -1:
46         print(0); print(1); return
47
48     # --- GLOBAL SETTINGS ---
49     best_path = []
50     max_len = -1
51     start_time = time.time()
52     DFS_TIME_LIMIT = 4.85
53     dfs_steps = 0
54
55     # --- LOCAL SEARCH OPTIMIZER ---
56     def optimize_full(path):
57         current_path = list(path)
58         path_mask = 0
59         for x in current_path: path_mask |= (1 << x)
60
61         has_improvement = True
62         while has_improvement:
63             has_improvement = False
64
65             # Phase 1: Triangle Insertion
66             i = 0
67             while i < len(current_path) - 1:
68                 u, v = current_path[i], current_path[i+1]
69                 # Bitmask check: neighbors of u AND neighbors of v AND not
used
70                 candidates = (adj_mask[u] & adj_mask[v]) & (~path_mask)
71
72                 if candidates > 0:

```

```

73         k = (candidates & -candidates).bit_length() - 1
74         current_path.insert(i + 1, k)
75         path_mask |= (1 << k)
76         has_improvement = True
77     else:
78         i += 1
79
80     if has_improvement: continue
81
82     # Phase 2: Box Insertion (u -> a -> b -> v)
83     unused_nodes = []
84     for node in range(N):
85         if not ((path_mask >> node) & 1): unused_nodes.append(node)
86
87     if len(unused_nodes) < 2: continue
88
89     i = 0
90     while i < len(current_path) - 1:
91         u, v = current_path[i], current_path[i+1]
92         inserted = False
93
94         # Candidates A connected to U
95         mask_A = adj_mask[u] & (~path_mask)
96         if not mask_A:
97             i += 1; continue
98
99         # Candidates B connected to V
100        mask_B = adj_mask[v] & (~path_mask)
101        if not mask_B:
102            i += 1; continue
103
104        # Check pairs
105        temp_A = mask_A
106        while temp_A > 0:
107            a = (temp_A & -temp_A).bit_length() - 1
108            temp_A &= temp_A - 1
109
110            temp_B = mask_B
111            while temp_B > 0:
112                b = (temp_B & -temp_B).bit_length() - 1
113                temp_B &= temp_B - 1
114
115                if a != b and ((adj_mask[a] >> b) & 1):
116                    current_path.insert(i+1, b)
117                    current_path.insert(i+1, a)
118                    path_mask |= (1<<a) | (1<<b)
119                    has_improvement = True
120                    inserted = True
121                    break
122            if inserted: break
123
124        if not inserted: i += 1
125
126    return current_path
127
128    # --- BRANCH AND BOUND (LOOKAHEAD) ---
129    def can_reach_target(u, current_mask):
130        if u == target: return True

```

```

131     if dist_from_target[u] == -1: return False
132     q_check = [u]
133     seen_mask = (1 << u)
134     while q_check:
135         curr = q_check.pop(0)
136         if curr == target: return True
137         for v in adj[curr]:
138             if not ((current_mask >> v) & 1) and not ((seen_mask >> v)
& 1):
139                 if v == target: return True
140                 seen_mask |= (1 << v)
141                 q_check.append(v)
142     return False
143
144 # --- BACKTRACKING DFS ---
145 def dfs(u, path_nodes, visited_mask, w_deg, w_dist, chaos_prob):
146     nonlocal max_len, best_path, dfs_steps
147
148     dfs_steps += 1
149     if dfs_steps & 63 == 0:
150         if time.time() - start_time > DFS_TIME_LIMIT: return True
151
152     if u == target:
153         if len(path_nodes) >= max_len:
154             max_len = len(path_nodes)
155             best_path = list(path_nodes)
156         return False
157
158     # Loc ung vien hop le (Pruning)
159     raw_neighbors = [v for v in adj[u] if not ((visited_mask >> v) & 1)
]
160     valid_neighbors = []
161     for v in raw_neighbors:
162         if v == target: valid_neighbors.append(v)
163         else:
164             if can_reach_target(v, visited_mask | (1 << v)):
165                 valid_neighbors.append(v)
166
167     if not valid_neighbors: return False
168
169     # Heuristic Scoring
170     scored_neighbors = []
171     for v in valid_neighbors:
172         if v == target: score = 10**9
173         else:
174             deg = 0
175             for next_v in adj[v]:
176                 if not ((visited_mask >> next_v) & 1): deg += 1
177             dst = dist_from_target[v]
178             score = (deg * w_deg) - (dst * w_dist) + random.uniform
(-2.0, 2.0)
179             scored_neighbors.append((score, v))
180
181     scored_neighbors.sort(key=lambda x: x[0])
182
183     # Chaos Swap (Randomized)
184     if len(scored_neighbors) > 1 and chaos_prob > 0:
185         if not (scored_neighbors[-1][1] == target and len(

```

```

186         scored_neighbors) == 2):
187             if random.random() < chaos_prob:
188                 scored_neighbors[0], scored_neighbors[1] =
189                 scored_neighbors[1], scored_neighbors[0]
190
191         for _, v in scored_neighbors:
192             if dfs(v, path_nodes + [v], visited_mask | (1 << v), w_deg,
193                 w_dist, chaos_prob): return True
194             if dfs_steps & 63 == 0 and time.time() - start_time >
195                 DFS_TIME_LIMIT: return True
196
197         return False
198
199     # --- MAIN LOOP (RESTART STRATEGY) ---
200     iteration = 0
201     while True:
202         if time.time() - start_time > DFS_TIME_LIMIT: break
203         iteration += 1
204
205         # Dynamic Weights
206         if iteration < 80:
207             w_d = random.uniform(5.0, 20.0); w_dst = random.uniform(0.5,
208             5.0); chaos = 0.1
209         else:
210             mode = iteration % 3
211             if mode == 0: w_d, w_dst, chaos = 25.0, 0.5, 0.05
212             elif mode == 1: w_d, w_dst, chaos = 5.0, 10.0, 0.05
213             else: w_d, w_dst, chaos = 10.0, 2.0, 0.4
214
215         try:
216             dfs(start_node, [start_node], 1 << start_node, w_d, w_dst,
217             chaos)
218         except RecursionError: pass
219
220     # --- FINAL OUTPUT ---
221     if max_len != -1:
222         final_path = optimize_full(best_path) # Chay Local Search lan cuoi
223         print(len(final_path) - 1)
224         print(*(x + 1 for x in final_path))
225     else:
226         print(0); print(1)
227
228 if __name__ == '__main__':
229     solve()

```

Listing 1: Giải thuật Hybrid Randomized DFS kết hợp Local Search

2 Lời giải bài B: Phân ca nhân viên

2.1 Chiến lược thiết kế thuật toán: Transform and Conquer

Bài toán được giải quyết dựa trên tư tưởng Transform and Conquer (Biến đổi và Tri). Cụ thể, ta áp dụng hai hình thức:

1. **Problem Reduction (Biến đổi sang bài toán đã biết):** Ta chuyển đổi bài toán "phân chia ca làm việc" ban đầu thành bài toán 2-Satisfiability (2-SAT) - một bài toán quyết định sự thỏa mãn của biểu thức logic.
2. **Representation Change (Thay đổi biểu diễn):** Để giải quyết bài toán 2-SAT hiệu quả, ta không xử lý trực tiếp trên các mệnh đề logic mà chuyển đổi (transform) biểu diễn của chúng sang Đồ thị dẫn xuất (Implication Graph).

Quá trình áp dụng cụ thể:

- **Transform (Biến đổi):**

- Mỗi nhân viên i được đại diện bởi biến logic a_i (*True*: ca sáng, *False*: ca chiều).
- Các ràng buộc được chuyển thành các mệnh đề dạng $(u \vee v)$.
- Dựa trên tính chất logic $(u \vee v) \Leftrightarrow (\neg u \rightarrow v) \wedge (\neg v \rightarrow u)$, ta xây dựng đồ thị có hướng với các đỉnh là a_i và $\neg a_i$. Mỗi mệnh đề sẽ tương ứng với 2 cạnh trên đồ thị.

- **Conquer (Tri):** Áp dụng thuật toán Tarjan để tìm các Thành phần liên thông mạnh (Strongly Connected Components - SCC) trên đồ thị vừa xây dựng. Đây là bài toán đồ thị kinh điển đã có thuật toán giải hiệu quả, giải quyết bằng phương pháp quy hoạch động (dynamic programming) thông qua thuật toán Tarjan.
- **Kết luận (Tổng hợp):** Nếu tồn tại một biến x mà cả x và $\neg x$ cùng thuộc một thành phần liên thông mạnh (tức là $x \rightarrow \neg x$ và $\neg x \rightarrow x$), bài toán vô nghiệm (YES). Ngược lại, ta luôn tìm được cách phân ca hợp lệ (NO).

2.2 Phân tích chi tiết

Ta có M ràng buộc, gồm 2 loại:

- Loại 1: nhân viên i và j phải làm cùng ca hay a_i và a_j cùng giá trị, cụ thể là $(\neg a_i \vee a_j) \wedge (a_i \vee \neg a_j)$.
- Loại 2: nhân viên i và j phải làm khác ca hay a_i và a_j khác giá trị, cụ thể là $(a_i \vee a_j) \wedge (\neg a_i \vee \neg a_j)$.

Ta mô hình hoá: với mỗi a_i sẽ có 2 đỉnh là a_i và $\neg a_i$. Một điều kiện $(u \vee v) \Leftrightarrow (\neg u \rightarrow v)$ là cạnh nối từ $\neg u$ đến v , tương tự $(u \vee v) = (v \vee u) \Leftrightarrow (\neg v \rightarrow u)$ là cạnh nối từ $\neg v$ đến u . Nên với mỗi điều kiện như trên ta sẽ thêm 2 cạnh vào đồ thị.

Sau đó ta nhận xét rằng trong một thành phần liên thông mạnh thì các đỉnh phải có cùng giá trị chân lý, theo tính chất của bài toán **2-SAT**. Ta sử dụng thuật toán **Tarjan** để tìm ra các SCC này.

Điều kiện tồn tại nghiệm: a_i và $\neg a_i$ phải nằm khác thành phần liên thông mạnh. Nếu chúng nằm cùng một SCC, nghĩa là $a_i \Rightarrow \neg a_i$ và $\neg a_i \Rightarrow a_i$, dẫn đến mâu thuẫn logic.

2.2.1 Phân tích độ phức tạp thời gian:

- 1. Xác định thao tác cơ bản: Phép tính cập nhật mảng **low** và duyệt cạnh trong thuật toán **Tarjan/DFS**.
- 2. Số lần thao tác cơ bản được thực hiện: Mỗi đỉnh ta chỉ duyệt qua một lần nên tổng $2n$ lần thực hiện thao tác (do đồ thị có $2n$ đỉnh). Tại mỗi đỉnh chỉ cần duyệt qua các cạnh kề của nó. Mỗi ràng buộc tạo ra 4 cạnh (hoặc 2 mệnh đề, mỗi mệnh đề 2 cạnh), tổng số cạnh là $O(m)$.
- 3. Thiết lập hàm thời gian: $T(n, m) \approx V + E = 2n + 4m$.
- 4. Lấy Big-O: $O(n + m)$.

2.2.2 Phân tích độ phức tạp không gian:

- 1. Xác định các vùng nhớ được sử dụng: Ta sử dụng danh sách kề *adj* để lưu đồ thị và các mảng phụ trợ (*ids*, *low*, *on_stack*) kích thước $2n$.
- 2. Tính tổng bộ nhớ:
 - Đỉnh: $O(2n)$
 - Cạnh: $O(4m)$
- 3. Thiết lập hàm không gian $S(n, m) = O(n + m)$.
- 4. Lấy Big-O: $O(n + m)$.

2.2.3 Code python:

```
1 import sys
2
3 sys.setrecursionlimit(2000)
4
5 def solve():
6     input_data = sys.stdin.read().split()
7     if not input_data: return
8     iterator = iter(input_data)
9
10    try:
11        n = int(next(iterator))
12        m = int(next(iterator))
13    except StopIteration: return
14
15    limit = 2 * n + 1
16
17    adj = [[] for _ in range(limit)]
18
19    for _ in range(m):
20        t = int(next(iterator))
21        u = int(next(iterator))
22        v = int(next(iterator))
23
24        not_u = u + n if u <= n else u - n
25        not_v = v + n if v <= n else v - n
```

```

26
27     if t == 2:
28         adj[not_u].append(v)
29         adj[not_v].append(u)
30         adj[u].append(not_v)
31         adj[v].append(not_u)
32     else:
33         adj[u].append(v)
34         adj[not_v].append(not_u)
35         adj[not_u].append(not_v)
36         adj[v].append(u)
37
38
39     ids = [0]    limit
40     low = [0]    limit
41     num = [0]    limit
42     on_stack = [False]    limit
43
44     st = []
45     work = []
46
47     ptr = [0]    limit
48
49     timer = 0
50     scc_count = 0
51
52     for start_node in range(1, limit):
53         if num[start_node]:
54             continue
55
56         work.append(start_node)
57
58         while work:
59             u = work[-1]
60
61             if num[u] == 0:
62                 timer += 1
63                 num[u] = low[u] = timer
64                 st.append(u)
65                 on_stack[u] = True
66
67             found_new_child = False
68
69             while ptr[u] < len(adj[u]):
70                 v = adj[u][ptr[u]]
71                 ptr[u] += 1
72
73                 if num[v] == 0:
74                     work.append(v)
75                     found_new_child = True
76                     break
77                 elif on_stack[v]:
78                     if num[v] < low[u]:
79                         low[u] = num[v]
80
81             if found_new_child:
82                 continue
83

```

```

84         work.pop()
85
86     if work:
87         parent = work[-1]
88         if low[u] < low[parent]:
89             low[parent] = low[u]
90
91     if low[u] == num[u]:
92         scc_count += 1
93         while True:
94             node = st.pop()
95             on_stack[node] = False
96             ids[node] = scc_count
97             if node == u:
98                 break
99
100     answer = True
101     for i in range(1, n + 1):
102         if ids[i] == ids[i + n]:
103             answer = False
104             break
105
106     sys.stdout.write("YES" if not answer else "NO")
107
108 if __name__ == '__main__':
109     solve()

```